

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Операционные системы»
Тема: Межпроцессные взаимодействия,
IPC (sig, pipe, fifo, msg)

Студентка гр. 2304

Иванова М.А.

Преподаватель

Душутина Е.В.

Санкт-Петербург

2024

Цель работы.

Изучить межпроцессные взаимодействия в Unix-подобных ОС. Изучить команды и утилиты для межпроцессных взаимодействий Linux (в частности, sig, pipe, fifo, msg), научиться применять их на практике.

Задание.

1. Используя функцию `sigaction()`, продемонстрируйте возможности управления линейкой сигналов, включая собственные обработчики и маскирование для разных сигналов, а также вариативность, предоставляемую структурами `sigaction (act / oldact)`.
2. Реализуйте **надежные** сигналы, организуйте эксперимент для доказательства отложенной обработки (например, в случае поступления сигнала во время уже выполняющейся обработки другого сигнала). Сгенерируйте ситуацию с несколькими отложенными сигналами.
3. Экспериментально продемонстрируйте **разницу** между надежными и **ненадежными** сигналами.
4. Организуйте «вложенные» надежные сигналы, для этого из обработчика одного сигнала необходимо произвести отправку другого сигнала, а также отправку такого же сигнала.
5. Проведите эксперимент, доказывающий возможность организации *очереди для различных типов сигналов, обычных и сигналов реального времени*;
Проверьте возможность организации очереди для «вложенных» сигналов PV.
6. Опытным путем подтвердите наличие **приоритетов сигналов** реального времени.
7. Экспериментально подтвердите, что обработка равно-приоритетных сигналов реального времени происходит в порядке FIFO.

Выполнение работы.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ neofetch
...-::-:-...
.-MMMMMMMMMMMMMMMMM-.
.-MMM`.-:::-:-..`MMM-.
.:MMMM.:MMMMMMMMMMMMMMMM:.MMM:.
-MMM-M--MMMMMMMMMMMMMMMMMMMM.MMM-
`:MMM:MM` :MMM:.....-.-MMM:MM:`
:MMM:MM` :MM: ` ` ` ` :MMM:MM:
.MMM.MMMM` :MM. -MM. .MM- `MMMM.MMM.
:MM:M MMM` :MM. -MM- .MM: `MMMM-MMM:
:MM:M MMM` :MM. -MM- .MM: `MMMM-MMM:
:MM:M MMM` :MM. -MM- .MM: `MMMM-MMM:
:MM:M MMM- `MMMMMMMMMMMMMM-` -MMM-MMM:
:MM:M MM: ` ` ` ` :MM:M MM:
.MMM.MMMM:-----:MMM.MMM.
'-MMM.-MMMMMMMMMMMMMMMM-.MMM-'
'..MMM`-:::-:-..`MMM-.'
'-MMMMMMMMMMMMMM-'
`-:::-:-`
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P
-----
OS: Linux Mint 21.3 x86_64
Host: 82JD Lenovo Legion Y9000P2021H
Kernel: 5.15.0-100-generic
Uptime: 3 hours, 36 mins
Packages: 2507 (dpkg), 8 (flatpak)
Shell: bash 5.1.16
Resolution: 2928x1830
DE: Cinnamon 6.0.4
WM: Mutter (Muffin)
WM Theme: Mint-L-Dark-Purple (Mint-Y
Theme: Mint-L-Dark-Purple [GTK2/3]
Icons: Mint-L-Purple [GTK2/3]
Terminal: gnome-terminal
CPU: 11th Gen Intel i7-11800H (16) @
GPU: Intel TigerLake-H GT1 [UHD Grap
GPU: NVIDIA GeForce RTX 3060 Mobile
Memory: 6176MiB / 31875MiB
```

Рис. 1 — Информация о системе

1. **Задание:** Используя функцию *sigaction()*, продемонстрируйте возможности управления линейкой сигналов, включая собственные обработчики и маскирование для разных сигналов, а также вариативность, предоставляемую структурами *sigaction (act / oldact)*.

Решение:

Функция `int sigaction( int sig, const struct sigaction * act, struct sigaction * oact)` позволяет вызывающему процессу получить информацию или

установить (или и то и другое) действие, соответствующее какому-либо сигналу или группе сигналов.

Sig - тип сигнала, act – ненулевое значение отвечает за изменение действия при указанном сигнале, oact - ненулевое значение сохраняет предыдущее действие при указанном сигнале в структуре типа sigaction, на которую указывает указатель oact. Комбинация act и oact позволяет запрашивать или устанавливать новые действия при поступлении сигнала.

Sigaction() гарантирует надежную передачу, т.е. если при возникновении сигнала система занята обработкой другого сигнала (назовем его «текущим»), то возникший сигнал не будет потерян, а его обработка будет отложена до окончания текущего обработчика.

Была написана программа, передопределяющая обработчики сигналов SIGINT и SIGQUIT, маскирующая сигналы SIGQUIT и SIGUSR1, вызывающая эти три сигнала, выводящая информацию о них и статус процессов.

Исходный код: Файл 1.c

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Обработчик сигнала для SIGINT
void handlerSIGINT(int signum) {
    printf("Обработчик SIGINT\n");
    system("ps -s");
    exit(0);
}

// Обработчик сигнала для SIGQUIT
```

```

void handlerSIGQUIT(int signum) {
    printf("Обработчик SIGQUIT\n");
}

int main() {
    // Структура для старого обработчика сигнала
    struct sigaction old_act;
    // Структура для нового обработчика сигнала
    struct sigaction act;
    // Сброс маски сигналов
    sigemptyset(&act.sa_mask);
    // Не используем специальных флагов при обработке сигнала
    act.sa_flags = 0;

    sigset_t mask1;
    sigset_t mask2;
    sigemptyset(&mask1);
    sigemptyset(&mask2);

    // Установка обработчика сигнала для SIGINT
    // Задание пользовательской функции обработки сигнала
    act.sa_handler = handlerSIGINT;
    // Связываем обработчик сигнала SIGINT с обработчиком act и old_act
    // В случае ошибки
    if (sigaction(SIGINT, &act, &old_act) == -1) {
        printf("Ошибка sigaction\n");
        exit(1);
    }

    // Установка обработчика сигнала для SIGQUIT
    // Задание пользовательской функции обработки сигнала
    act.sa_handler = handlerSIGQUIT;
    // Связываем обработчик сигнала SIGQUIT с обработчиком act
    // В случае ошибки
    if (sigaction(SIGQUIT, &act, NULL) == -1) {
        printf("Ошибка sigaction\n");
        exit(1);
    }

    // Маскирование SIGQUIT (4)
    // добавляем SIGQUIT в маску
    sigaddset(&mask1, SIGQUIT);
    // сигналы в маске будут заблокированы
    if (sigprocmask(SIG_BLOCK, &mask1, NULL) == -1) {
        printf("Ошибка sigprocmask\n");
        exit(1);
    }

    // Маскирование SIGUSR1 (200)
    // добавляем SIGUSR1 в маску
    sigaddset(&mask2, SIGUSR1);
    if (sigprocmask(SIG_BLOCK, &mask2, NULL) == -1) {

```

```

        printf("Ошибка sigprocmask\n");
        exit(1);
    }

    // Вывод информации об обработчиках сигналов
    printf("SIGINT: обработчик %p; старый обработчик %p, маска NULL\n",
handlerSIGINT, old_act.sa_handler);
    printf("SIGQUIT: обработчик %p; старый обработчик NULL; маска ",
handlerSIGQUIT);
    for (int i = 1; i <= 32; i++) {
        if (sigismember(&mask1, i)) {
            printf("%d ", i);

        }
    }
    printf("\n");
    printf("SIGUSR1: обработчик SIG_DFL; старый обработчик NULL; маска ");
    for (int i = 1; i <= 32; i++) {
        if (sigismember(&mask2, i)) {
            printf("%d ", i);

        }
    }
    printf("\n");

/*    // Вызываем сигналы с задержкой
    printf("\nВызов сигнала SIGQUIT\n");
    kill(getpid(), SIGQUIT);
    system("ps -s");
    sleep(1);*/

    printf("\nВызов сигнала SIGUSR1\n");
    kill(getpid(), SIGUSR1);
    system("ps -s");
    sleep(1);

    printf("\nВызов сигнала SIGINT\n");
    kill(getpid(), SIGINT);
    sleep(1);

    return 0;
}

```

Вывод программы:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/1$ ./1
```

```
SIGINT: обработчик 0x55bbc33032e9; старый обработчик (nil), маска NULL
```

SIGQUIT: обработчик 0x55bbc3303320; старый обработчик NULL; маска 3

SIGUSR1: обработчик SIG_DFL; старый обработчик NULL; маска 10

Вызов сигнала SIGQUIT

UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND

```
1000 14203 0000000000000000 0000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
1000 14706 0000000000000000 0000000000010204 0000000000000006 0000000000000000 S+ pts/0 0:00 ./1
1000 14707 0000000000000000 0000000000000000 0000000180000000 0000000000010002 S+ pts/0 0:00 sh -c ps -s
1000 14708 0000000000000000 0000000000000000 0000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s
```

Вызов сигнала SIGUSR1

UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND

```
1000 14203 0000000000000000 0000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
1000 14706 00000000000000200 0000000000010204 0000000000000006 0000000000000000 S+ pts/0 0:00 ./1
1000 14709 0000000000000000 0000000000000000 0000000180000000 0000000000010002 S+ pts/0 0:00 sh -c ps -s
1000 14710 0000000000000000 0000000000000000 0000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s
```

Вызов сигнала SIGINT

Обработчик SIGINT

UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND

```
1000 14203 0000000000000000 0000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
1000 14706 00000000000000200 0000000000010206 0000000000000006 0000000000000000 S+ pts/0 0:00 ./1
1000 14711 0000000000000000 0000000000000000 0000000180000000 0000000000010002 S+ pts/0 0:00 sh -c ps -s
1000 14712 0000000000000000 0000000000000000 0000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s
```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/1\$

Опытным путем было выяснено, что сигналу SIGUSR1 соответствует значение «200», а SIGQUIT - «4» в столбцах вывода статуса процессов. В столбце PENDING видны оба этих числа, что говорит о том, что с помощью маскирования SIGQUIT и SIGUSR1 были действительно заблокированы.

Отсутствие вывода сообщения о запуске обработчика SIGQUIT также подтверждает этот факт. А сработавший пользовательский обработчик SIGINT вывел соответствующее сообщение и завершил работу программы.

Кроме того, в выводе указаны адреса пользовательских обработчиков SIGINT и SIGQUIT, адрес предыдущего обработчика SIGINT и маски SIGQUIT и SIGUSR1. Значения, помеченные NULL, говорят о том, что эти поля просто не рассматривались в программе, но были оставлены для красоты вывода.

2. Задание: Реализуйте надежные сигналы, организуйте эксперимент для доказательства отложенной обработки (например, в случае поступления сигнала во время уже выполняющейся обработки другого сигнала). Сгенерируйте ситуацию с несколькими отложенными сигналами.

Решение:

Была написана программа, организующая надежные сигналы с помощью `sigaction()`: пользовательский обработчик сигнала SIGUSR1 вызывает временную блокировку (т.е. отложенную обработку) сигнала SIGINT посредством вызова `sigaddset(&act.sa_mask, SIGINT)`. При обработке сигнала предусмотрена задержка в несколько десятков секунд для наглядности работы кода. При запуске в фоновом режиме программа ждет отправки сигналов пользователем через терминал.

Исходный код: Файл 2.c

```
#include <stdio.h>
#include <signal.h>
```



```

#include <unistd.h>

// Надежный обработчик сигналов
void (*mysig(int signum, void (*handler)(int)))(int) {
    // Создание структуры для обработчика сигнала
    struct sigaction act;
    // Задание переданной пользовательской функции обработки сигнала
    act.sa_handler = handler;
    // Очищаем сигналы act.sa_mask, чтобы никакие другие сигналы не были
    // заблокированы во время обработки текущего
    sigemptyset(&act.sa_mask);
    // Добавляем сигнал SIGINT в act.sa_mask, чтобы заблокировать сигнал
    // SIGINT на время обработки текущего сигнала
    sigaddset(&act.sa_mask, SIGINT);
    sigaddset(&act.sa_mask, SIGUSR1);
    // Не используем специальных флагов при обработке сигнала
    act.sa_flags = 0;
    // Вызываем надежный обработчик сигнала signum с обработчиком act для
    // их связывания
    // В случае ошибки
    if (sigaction(signum, &act, 0) < 0) {
        // Вывести сообщение об ошибке
        return SIG_ERR;
    }
    // Возвращаем указатель на функцию обработки сигнала
    return act.sa_handler;
}

// Обработчик сигнала SIGUSR1
void handlerUSR1(int sig) {
    if (sig == SIGUSR1) {
        // Выводим сообщение
        printf("Пойман сигнал SIGUSR1\n");
    } else {
        // Выводим сообщение
        printf("Получен сигнал %d, отличный от SIGUSR1\n", sig);
        return;
    }
    // Блокируем сигнал SIGUSR1, вызвав "засыпание" на 40 секунд
    sleep(40);
}

int main() {
    // Вызываем надежную обработку сигнала
    mysig(SIGUSR1, handlerUSR1);
    while (1) {
        // Ставим программу на паузу
        // В случае получения сигнала SIGINT (Ctrl+C) программа завершится
        pause();
    }
    return 0;
}

```

```
}
```

Вывод программы (лог разделен комментариями):

Как станет дальше понятно из логов, “200” соответствует сигналу SIGUSR1, а “2” - сигналу SIGINT.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ gcc 2.c -o 2
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ./2 &
[1] 26029
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26029	00000000	00000000	00000000	00000200	S	pts/0	0:00	./2
1000	26032	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Сначала у программы нет ни отложенных, ни заблокированных, ни проигнорированных сигналов, есть только пойманный сигнал SIGUSR1 (“200”) вероятно потому, что в головной функции был программно вызван обработчик этого сигнала.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ kill -SIGUSR1 %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ Пойман сигнал
SIGUSR1
jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26029	00000000	00000202	00000000	00000200	S	pts/0	0:00	./2
1000	26033	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

После принятия первого сигнала SIGUSR1 из консоли меняется статус сигналов - заблокированы сигналы SIGUSR1 и SIGINT. SIGINT

заблокирован из-за строки кода `sigaddset(&act.sa_mask, SIGINT)` в надежном обработчике сигнала `mysig`. Я думаю, что и сигнал `SIGUSR1` оказался в списке заблокированных благодаря использованию надежного обработчика сигналов `sigaction()`.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ kill -SIGUSR1 %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26029	00000200	00000202	00000000	00000200	S	pts/0	0:00	./2
1000	26034	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Не дожидаясь завершения обработки первого сигнала `SIGUSR1`, был отправлен второй сигнал `SIGUSR1`. Видно, что он отмечен в столбце `PENDING`.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ Пойман сигнал
SIGUSR1
jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26029	00000000	00000202	00000000	00000200	S	pts/0	0:00	./2
1000	26045	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

После обработки первого сигнала `SIGUSR1`, начинается обработка второго сигнала `SIGUSR1` - отметка “200” исчезла из столбца `PENDING`.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ kill -SIGUSR1 %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash

```

1000 26029 00000200 00000202 00000000 00000200 S pts/0 0:00 ./2
1000 26048 00000000 00000000 00000000 73d1fef9 R+ pts/0 0:00 ps -s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ kill -SIGINT %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ jobs
[1]+  Running                  ./2 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26029	00000202	00000202	00000000	00000200	S	pts/0	0:00	./2
1000	26049	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Отправим сигнал SIGUSR1 в третий раз и сразу же отправим сигнал SIGINT. Программа продолжает свою работу, несмотря на сигнал прерывания. Дело в том, что сигналы надежные - обработка одного сигнала (SIGUSR1) не прерывается при возникновении другого (SIGINT). Оба сигнала видны в листе ожидания PENDING.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ jobs
[1]+  Interrupt                ./2
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	25672	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	26066	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/2$

```

Через какое-то время, достаточное для обработки второго сигнала SIGUSR1, но недостаточное для обработки третьего сигнала SIGUSR1, программа завершила свою работу. Это говорит о том, что сигнал SIGINT в очереди на обработку стоит выше, чем сигнал SIGUSR1. Можно предположить, порядок обработки сигналов в очереди соответствует их порядковому номеру. Действительно, сигнал SIGINT (“2”) был обработан раньше, чем сигнал SIGUSR1 (“200”).

Таким образом, программа демонстрирует работу надежных сигналов - сигналы, поступившие во время обработки текущего сигнала, не прерывают ее, а добавляются в очередь.

3. Экспериментально продемонстрируйте разницу между надежными и ненадежными сигналами.

Работа надежных сигналов продемонстрирована в предыдущем задании. Для демонстрации работы ненадежных сигналов была написана программа, которая вызывает ненадежный обработчик сигнала SIGUSR1 с помощью signal().

Исходный код: Файл 3.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// Обработчик сигнала SIGUSR1
void handlerUSR1(int sig) {
    if (sig == SIGUSR1) {
        // Выводим сообщение
        printf("Пойман сигнал SIGUSR1\n");
    }
    // Задерживаем обработку сигнала на 20 секунд
    sleep(20);
}

int main() {
    // Вызываем ненадежную обработку сигнала
    signal(SIGUSR1, handlerUSR1);
    while (1) {
        // Ставим программу на паузу
        // В случае получения сигнала SIGINT (Ctrl+C) программа
        // завершится
        pause();
    }
}
```

```

}
return 0;
}

```

Результат

```

krl@krl-TM1703:~/Desktop/2-3$ ./signal
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ gcc 3.c -o 3
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ ./3 &
[1] 29087
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	28438	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	29087	00000000	00000000	00000000	00000200	S	pts/0	0:00	./3
1000	29088	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Поведение текущей программы с ненадежными сигналами в данной точке

совпадает с работой предыдущей программы с надежными сигналами.

Анализ соответствующий.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ kill -SIGUSR1 %1
Пойман сигнал SIGUSR1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	28438	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	29087	00000000	00000000	00000000	00000200	S	pts/0	0:00	./3
1000	29091	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Здесь начинаются различия: в столбце **BLOCKED** нет никаких сигналов, что говорит о том, что ненадежный обработчик сигнала `signal()` не добавляет в список блокируемых вызываемый сигнал `SIGUSR1`. Вывод сообщения пользовательским обработчиком говорит о том, что сигнал начал обрабатываться.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ kill -SIGUSR1 %1
Пойман сигнал SIGUSR1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	28438	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	29087	00000000	00000000	00000000	00000200	S	pts/0	0:00	./3
1000	29093	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Еще раз вызовем сигнал SIGUSR1 до конца обработки первого сигнала

SIGUSR1. Вывод сообщения пользовательским обработчиком говорит о том, что второй сигнал SIGUSR1 все равно принят в обработку.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ kill -SIGINT %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	28438	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	29094	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

[1]+ Interrupt . /3

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/3$
```

Теперь отправим сигнал SIGINT во время работы обработчика второго сигнала SIGUSR1. Работа программы тут же завершается: вместо завершения обработки сигнала SIGUSR1 была произведена обработка сигнала SIGINT.

Таким образом, программа демонстрирует работу ненадежных сигналов - сигналы, поступившие во время обработки текущего сигнала, прерывают ее, начиная обработку последнего поступившего сигнала.

В этом и заключается отличие надежной обработки сигналов от ненадежной: у надежных сигналов есть возможность отложить прием некоторых других сигналов (за счет маски sa_mask в структуре sigaction), тогда как у ненадёжных сигналов это не представляется возможным.

4. Задание: Организуйте «вложенные» надежные сигналы, для этого из обработчика одного сигнала необходимо произвести отправку другого сигнала, а также отправку такого же сигнала.

Решение:

Была написана программа, организующая «вложенные» надежные сигналы с помощью `sigaction()`: программа вызывает временную блокировку (т.е. отложенную обработку) сигнала `SIGINT` посредством вызова `sigaddset(&act.sa_mask, SIGINT)` и отправку сигнала `SIGINT` и `SIGUSR1` из пользовательского обработчика сигнала `SIGUSR1`, а также отправку сигнала `SIGINT` и `SIGUSR1` из пользовательского обработчика. При обработке сигнала предусмотрена задержка в несколько десятков секунд для наглядности работы кода. При запуске в фоновом режиме программа ждет отправки сигналов пользователем через терминал.

Исходный код: Файл 4.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// Надежный обработчик сигналов
void (*mysig(int signum, void (*handler)(int)))(int) {
    // Создание структуры для обработчика сигнала
    struct sigaction act;
    // Задание переданной пользовательской функции обработки сигнала
    act.sa_handler = handler;
    // Очищаем сигналы act.sa_mask, чтобы никакие бругие сигналы не были
    // заблокированы во время обработки текущего
    sigemptyset(&act.sa_mask);
```



```

    // Добавляем сигнал SIGINT в act.sa_mask, чтобы заблокировать сигнал
    SIGINT на время обработки текущего сигнала
    sigaddset(&act.sa_mask, SIGINT);
    // Не используем специальных флагов при обработке сигнала
    act.sa_flags = 0;
    // Вызываем надежный обработчик сигнала signum с обработчиком act для
    их связывания
    // В случае ошибки
    if (sigaction(signum, &act, 0) < 0) {
        // Вывести сообщение об ошибке
        return SIG_ERR;
    }
    // Возвращаем указатель на функцию обработки сигнала
    return act.sa_handler;
}

// Обработчик сигнала SIGUSR1
void handlerUSR1(int sig) {
    if (sig != SIGUSR1) {
        printf("Получен сигнал %d, отличный от SIGUSR1\n", sig);
        return;
    }
    // Выводим сообщение
    printf("Пойман сигнал SIGUSR1\n");

    sleep(5);
    printf("Отправляем сигнал SIGINT\n");
    // Генерируем сигнал SIGINT из текущего обработчика сигнала SIGUSR1
    kill(getpid(), SIGINT);

    sleep(5);
    printf("Отправляем сигнал SIGUSR1\n");
    // Генерируем сигнал SIGUSR1 из текущего обработчика сигнала SIGUSR1
    kill(getpid(), SIGUSR1);

    // Блокируем сигнал SIGUSR1, вызвав "засыпание" на 60 секунд
    sleep(40);
}

int main() {
    // Вызываем надежную обработку сигнала
    mysig(SIGUSR1, handlerUSR1);
}

```

```

while (1) {
    // Ставим программу на паузу
    // В случае получения сигнала SIGINT (Ctrl+C) программа завершится
    pause();
}
return 0;
}

```

Вывод программы (лог прерывается анализом его логической части):

Как станет дальше понятно из логов, “200” соответствует сигналу SIGUSR1, а “2” - сигналу SIGINT.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ gcc 4.c -o 4
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ./4 &
[1] 32900
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ jobs
[1]+  Running                  ./4 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32900	00000000	00000000	00000000	00000200	S	pts/0	0:00	./4
1000	32901	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Сначала у программы нет ни отложенных, ни заблокированных, ни проигнорированных сигналов, есть только пойманный сигнал SIGUSR1 (“200”) вероятно потому, что в головной функции был программно вызван обработчик этого сигнала.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ kill -SIGUSR1 %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ Пойман сигнал SIGUSR1
jobs
[1]+  Running                  ./4 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32900	00000000	00000202	00000000	00000200	S	pts/0	0:00	./4
1000	32902	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

После принятия первого сигнала SIGUSR1 из консоли меняется статус сигналов - заблокированы сигналы SIGUSR1 и SIGINT. SIGINT заблокирован из-за строки кода sigaddset(&act.sa_mask, SIGINT). Я думаю, что и сигнал SIGUSR1 оказался в списке заблокированных благодаря использованию надежного обработчика сигналов sigaction().

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ Отправляем сигнал SIGINT
```

```
jobs
```

```
[1]+  Running                  ./4 &
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32900	00000002	00000202	00000000	00000200	S	pts/0	0:00	./4
1000	32903	00000000	00000000	00000000	73dlfef9	R+	pts/0	0:00	ps -s

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ Отправляем сигнал SIGUSR1
```

```
jobs
```

```
[1]+  Running                  ./4 &
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s
```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32900	00000202	00000202	00000000	00000200	S	pts/0	0:00	./4
1000	32904	00000000	00000000	00000000	73dlfef9	R+	pts/0	0:00	ps -s

На двух частях лога выше видно работу пользовательского обработчика сигнала SIGUSR1: с задержкой в 5 секунд генерируются сигналы SIGINT и SIGUSR1 из обработчика, что отражается на статусах процессов: сначала к отложенным сигналам добавляется SIGINT, а потом и SIGUSR1. Обработка сгенерированных сигналов откладывается до конца выполнения текущего обработчика SIGUSR1, что иллюстрирует надежность сигналов.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ kill -SIGINT %1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ jobs
[1]+  Running                  ./4 &
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32900	00000202	00000202	00000000	00000200	S	pts/0	0:00	./4
1000	32910	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

После получения сигнала SIGINT из консоли программа по-прежнему продолжала работать в течение пары десятков секунд, потому что прием надежного сигнала SIGINT был отложен в пользовательской функции обработки сигнала.

По логам не видно внесение сигнала SIGINT в ожидающие сигналы (PENDING), потому что SIGINT присутствовал в маске сигналов и до этого.

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ jobs
[1]+  Interrupt                ./4
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/4$ ps -s

```

UID	PID	PENDING	BLOCKED	IGNORED	CAUGHT	STAT	TTY	TIME	COMMAND
1000	31686	00000000	00010000	00384004	4b813efb	Ss	pts/0	0:00	bash
1000	32919	00000000	00000000	00000000	73d1fef9	R+	pts/0	0:00	ps -s

Спустя несколько десятков секунд (из-за sleep()) в пользовательском обработчике сигнала SIGUSR1) после завершения обработки сигнала SIGUSR1 произошла обработка сигнала SIGINT, что привело к завершению работы программы, о чем свидетельствует лог.

Таким образом, программа демонстрирует работу вложенных надежных сигналов.

5. Задание: проведите эксперимент, позволяющий определить возможность организации очереди для различных типов сигналов, обычных

и реального времени, (более двух сигналов, для этого увеличьте «вложенность» вызовов обработчиков).

Решение:

Для проведения эксперимента написана программа.

В начале работы программа предлагает настроить свою работу: выбрать работу с сигналами реального времени / обычными сигналами; установить, блокируются ли сигналы при обработке другого или нет; установить, используется ли вложенная обработка сигналов.

После этого создается дочерний процесс, который устанавливает новые обработчики сигналов и ждёт получения сигналов.

Если выбран флаг блокировки сигналов, то в блок до установки обработчиков в маску *sa_mask* добавляются нужные сигналы.

Если выбран флаг вложенной обработки, то обработчик посылает другой сигнал своему же процессу с данными в 2 раза больше, чем получил.

Родитель ждёт с интервалом в секунду отправляет 6 сигналов потомку. Если не выбрана вложенность, то сигналы чередуются: SIGUSR1 и SIGUSR2 (в случае обычных сигналов) или SIGRTMIN и SIGRTMIN+1 (в случае если выбраны сигналы реального времени). Если вложенность выбрана, то посылается лишь один тип сигнала.

Исходный код задания: Файл 5.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
```

```

// Флаги выполнения
int rtFlag, blockFlag, nestedFlag = 0;

// Обработчик сигнала
void sigusrHandler(int sig, siginfo_t *info, void *ptr) {
    // Вывод информационного сообщения
    printf("Дочерний процесс поймал сигнал %d с данными %d.\n",
        sig, info->si_value.sival_int);
    // Если включена вложенность
    if (nestedFlag) {
        // Формирование данных
        union sigval val;
        val.sival_int = info->si_value.sival_int * 2;
        // Если включены сигналы PB
        if (rtFlag) {
            // Если сигнал был отправлен родителем -> отправить другой сигнал
            if (sig == SIGRTMIN) {
                sigqueue(getpid(), SIGRTMIN + 1, val);
            }
            // Иначе
        } else {
            // Если сигнал был отправлен родителем -> отправить другой сигнал
            if (sig == SIGUSR1) {
                sigqueue(getpid(), SIGUSR2, val);
            }
        }
    }
    // Приостановка на 3 секунды
    sleep(3);
    // Вывод информационного сообщения
    printf("Дочерний процесс закончил обработку сигнала %d с
данными %d!\n",
        sig, info->si_value.sival_int);
}

// Код потомка
void child() {
    // Вывод информационного сообщения
    printf("Дочерний процесс: pid = %d, ppid = %d\n", getpid(), getppid());
    // Формирование структуры sigaction
    struct sigaction act, oldact;
    // Обработчик сигнала

```

```

act.sa_sigaction = sigusrHandler;
// Установка маски и флагов
sigemptyset(&act.sa_mask);
act.sa_flags = SA_SIGINFO;
// Если используются сигналы PV
if (rtFlag) {
    // Если используется блокировка -> обновление маски
    if (blockFlag) {
        sigaddset(&act.sa_mask, SIGRTMIN);
        sigaddset(&act.sa_mask, SIGRTMIN + 1);
    }
    // Установка обработчиков
    sigaction(SIGRTMIN, &act, &oldact);
    sigaction(SIGRTMIN + 1, &act, &oldact);
}
// Иначе
else {
    // Если используется блокировка -> обновление маски
    if (blockFlag) {
        sigaddset(&act.sa_mask, SIGUSR1);
        sigaddset(&act.sa_mask, SIGUSR2);
    }
    // Установка обработчиков
    sigaction(SIGUSR1, &act, &oldact);
    sigaction(SIGUSR2, &act, &oldact);
}

// Вывод информационного сообщения
printf("Обработчики сигналов изменены дочерним процессом\n");
// Ожидание сигналов
while (1) {
    pause();
}

// Код родителя
void parent(pid_t child) {
    // Приостановка на 1 секунду
    sleep(1);
    // Вывод информационного сообщения
    printf("Отправка сигналов дочернему процессу\n\n");
    // Отправка 6 сигналов

```

```

for (int i = 0; i < 6; i++) {
    // Приостановка на 1 секунду
    sleep(1);
    // Формирование данных сигнала
    union sigval val;
    val.sival_int = i;
    // Если не используется ветвление
    if (!nestedFlag) {
        // Отправка сигналов PB / обычных сигналов
        if (rtFlag)
            sigqueue(child, SIGRTMIN + i % 2, val);
        else if (i % 2)
            sigqueue(child, SIGUSR2, val);
        else
            sigqueue(child, SIGUSR1, val);
    } else {
        // Отправка сигнала PB / обычного сигнала
        if (rtFlag)
            sigqueue(child, SIGRTMIN, val);
        else
            sigqueue(child, SIGUSR1, val);
    }
}
// Приостановка на 30 секунд
sleep(30);
// Вывод информационного сообщения
printf("Заканчиваем дочерний процесс\n\n");
// Остановка потомка
kill(child, SIGTERM);
exit(0);
}

int main() {
    // Буфер для символа
    char input;

    // Ввод флага сигналов PB
    printf("Использовать сигналы реального времени? [y/n] ");
    input = getchar();
    if (input == 'y')
        rtFlag = 1;
    getchar();
}

```



```

// Ввод флага блокировки
printf("Блокировать ли сигналы при обработке другого сигнала? [y/n] ");
input = getchar();
if (input == 'y')
    blockFlag = 1;
getchar();

// Ввод флага ветвления
printf("Использовать вложенную обработку сигналов? [y/n] ");
input = getchar();
if (input == 'y')
    nestedFlag = 1;

printf("\n");

// Вывод информационного сообщения
printf("Родительский процесс: pid = %d\n", getpid());

// Вызов fork() и переход к функциям
pid_t pid = fork();
if (pid == 0) {
    child();
} else {
    parent(pid);
}
}

```

Вывод программы:

Запуск без выбора флагов:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ gcc 5.c -o 5
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
Использовать сигналы реального времени? [y/n] n
Блокировать ли сигналы при обработке другого сигнала? [y/n] n
Использовать вложенную обработку сигналов? [y/n] n

Родительский процесс: pid = 31119

```

Дочерний процесс: pid = 31121, ppid = 31119
Обработчики сигналов изменены дочерним процессом
Отправка сигналов дочернему процессу

Дочерний процесс поймал сигнал 10 с данными 0.
Дочерний процесс поймал сигнал 12 с данными 1.
Дочерний процесс закончил обработку сигнала 12 с данными 1!
Дочерний процесс поймал сигнал 12 с данными 3.
Дочерний процесс закончил обработку сигнала 12 с данными 3!
Дочерний процесс поймал сигнал 12 с данными 5.
Дочерний процесс закончил обработку сигнала 12 с данными 5!
Дочерний процесс закончил обработку сигнала 10 с данными 0!
Дочерний процесс поймал сигнал 10 с данными 2.
Дочерний процесс закончил обработку сигнала 10 с данными 2!
Заканчиваем дочерний процесс

Запуск с флагом блокировки:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
Использовать сигналы реального времени? [y/n] n
Блокировать ли сигналы при обработке другого сигнала? [y/n] y
Использовать вложенную обработку сигналов? [y/n] n
```

Родительский процесс: pid = 32001
Дочерний процесс: pid = 32005, ppid = 32001
Обработчики сигналов изменены дочерним процессом
Отправка сигналов дочернему процессу

Дочерний процесс поймал сигнал 10 с данными 0.
Дочерний процесс закончил обработку сигнала 10 с данными 0!
Дочерний процесс поймал сигнал 10 с данными 2.
Дочерний процесс закончил обработку сигнала 10 с данными 2!
Дочерний процесс поймал сигнал 10 с данными 4.
Дочерний процесс закончил обработку сигнала 10 с данными 4!
Дочерний процесс поймал сигнал 12 с данными 1.
Дочерний процесс закончил обработку сигнала 12 с данными 1!
Заканчиваем дочерний процесс

Запуск с флагом блокировки и флагом вложенности:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
Использовать сигналы реального времени? [y/n] n
Блокировать ли сигналы при обработке другого сигнала? [y/n] y
Использовать вложенную обработку сигналов? [y/n] y
```

```
Родительский процесс: pid = 32028
Дочерний процесс: pid = 32029, ppid = 32028
Обработчики сигналов изменены дочерним процессом
Отправка сигналов дочернему процессу
```

```
Дочерний процесс поймал сигнал 10 с данными 0.
Дочерний процесс закончил обработку сигнала 10 с данными 0!
Дочерний процесс поймал сигнал 10 с данными 1.
Дочерний процесс закончил обработку сигнала 10 с данными 1!
Дочерний процесс поймал сигнал 10 с данными 3.
Дочерний процесс закончил обработку сигнала 10 с данными 3!
Дочерний процесс поймал сигнал 12 с данными 0.
Дочерний процесс закончил обработку сигнала 12 с данными 0!
Заканчиваем дочерний процесс
```

Запуск с флагом сигналов PV:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
Использовать сигналы реального времени? [y/n] y
Блокировать ли сигналы при обработке другого сигнала? [y/n] n
Использовать вложенную обработку сигналов? [y/n] n
```

```
Родительский процесс: pid = 32089
Дочерний процесс: pid = 32092, ppid = 32089
Обработчики сигналов изменены дочерним процессом
Отправка сигналов дочернему процессу
```

```
Дочерний процесс поймал сигнал 34 с данными 0.
```

Дочерний процесс поймал сигнал 35 с данными 1.
Дочерний процесс закончил обработку сигнала 35 с данными 1!
Дочерний процесс поймал сигнал 35 с данными 3.
Дочерний процесс закончил обработку сигнала 35 с данными 3!
Дочерний процесс поймал сигнал 35 с данными 5.
Дочерний процесс закончил обработку сигнала 35 с данными 5!
Дочерний процесс закончил обработку сигнала 34 с данными 0!
Дочерний процесс поймал сигнал 34 с данными 2.
Дочерний процесс закончил обработку сигнала 34 с данными 2!
Дочерний процесс поймал сигнал 34 с данными 4.
Дочерний процесс закончил обработку сигнала 34 с данными 4!
Заканчиваем дочерний процесс

Запуск с флагом сигналов PV и блокировки:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
Использовать сигналы реального времени? [y/n] y
Блокировать ли сигналы при обработке другого сигнала? [y/n] y
Использовать вложенную обработку сигналов? [y/n] n
```

Родительский процесс: pid = 32099
Дочерний процесс: pid = 32100, ppid = 32099
Обработчики сигналов изменены дочерним процессом
Отправка сигналов дочернему процессу

Дочерний процесс поймал сигнал 34 с данными 0.
Дочерний процесс закончил обработку сигнала 34 с данными 0!
Дочерний процесс поймал сигнал 34 с данными 2.
Дочерний процесс закончил обработку сигнала 34 с данными 2!
Дочерний процесс поймал сигнал 34 с данными 4.
Дочерний процесс закончил обработку сигнала 34 с данными 4!
Дочерний процесс поймал сигнал 35 с данными 1.
Дочерний процесс закончил обработку сигнала 35 с данными 1!
Дочерний процесс поймал сигнал 35 с данными 3.
Дочерний процесс закончил обработку сигнала 35 с данными 3!
Дочерний процесс поймал сигнал 35 с данными 5.
Дочерний процесс закончил обработку сигнала 35 с данными 5!
Заканчиваем дочерний процесс

Запуск со всеми флагами:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/5$ ./5
```

```
Использовать сигналы реального времени? [y/n] y
```

```
Блокировать ли сигналы при обработке другого сигнала? [y/n] y
```

```
Использовать вложенную обработку сигналов? [y/n] y
```

```
Родительский процесс: pid = 32111
```

```
Дочерний процесс: pid = 32112, ppid = 32111
```

```
Обработчики сигналов изменены дочерним процессом
```

```
Отправка сигналов дочернему процессу
```

```
Дочерний процесс поймал сигнал 34 с данными 0.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 0!
```

```
Дочерний процесс поймал сигнал 34 с данными 1.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 1!
```

```
Дочерний процесс поймал сигнал 34 с данными 2.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 2!
```

```
Дочерний процесс поймал сигнал 34 с данными 3.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 3!
```

```
Дочерний процесс поймал сигнал 34 с данными 4.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 4!
```

```
Дочерний процесс поймал сигнал 34 с данными 5.
```

```
Дочерний процесс закончил обработку сигнала 34 с данными 5!
```

```
Дочерний процесс поймал сигнал 35 с данными 0.
```

```
Дочерний процесс закончил обработку сигнала 35 с данными 0!
```

```
Дочерний процесс поймал сигнал 35 с данными 2.
```

```
Дочерний процесс закончил обработку сигнала 35 с данными 2!
```

```
Дочерний процесс поймал сигнал 35 с данными 4.
```

```
Дочерний процесс закончил обработку сигнала 35 с данными 4!
```

```
Дочерний процесс поймал сигнал 35 с данными 6.
```

```
Дочерний процесс закончил обработку сигнала 35 с данными 6!
```

```
Дочерний процесс поймал сигнал 35 с данными 8.
```

```
Дочерний процесс закончил обработку сигнала 35 с данными 8!
```

```
Дочерний процесс поймал сигнал 35 с данными 10.
```

```
Заканчиваем дочерний процесс
```

Как показали логи, из обычных сигналов невозможно создать очередь. Включение сигналов в маску блокировки не помогло улучшить результат. В каждом случае запуска было утеряно несколько сигналов. Это связано с тем, что полученный сигнал добавляется в маску принятых сигналов, и она не может хранить очередь принятых сигналов.

Из сигналов реального времени можно создать очередь. Для того чтобы избежать прерывания обработчика сигналов другим обработчиком, следует использовать блокирование сигналов. В этом случае сигналы с меньшим номером принимаются раньше, а сигналы с одним номером принимаются в порядке FIFO. При этом не происходит потери ни одного сигнала даже в случае вложенности.

6. Задание: Опытным путем подтвердите наличие приоритетов сигналов реального времени.

Решение:

Была написана программа, отправляющая несколько несколько сигналов реального времени. в разном порядке, добавив в обработчики задержку на случай если сигналы будут обрабатываться слишком быстро для формирования очереди.

Исходный код: Файл 6.c

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <wait.h>
```

```

#include <stdbool.h>
#include <stdlib.h>

// Обработчик сигналов
void handler(int signal, siginfo_t *info, void *context) {
    printf("Пойман сигнал PB: %d, pid = %d, ppid = %d\n", signal
- SIGRTMIN, getpid(), getppid());
    usleep(1000);
}

int main() {
    // Установка буферизации вывода в стандартный вывод на NULL,
что позволяет немедленно выводить информацию
    setbuf(stdout, NULL);
    // Создание дочернего процесса
    pid_t child = fork();
    if (child == 0) {
        struct sigaction act;

        // Установка обработчика сигналов
        act.sa_sigaction = handler;
        // Сброс флагов
        sigemptyset(&act.sa_mask); // Сброс флагов
        // Флаг дополнительной информации о сигнале
        act.sa_flags = SA_SIGINFO;

        for (int i = SIGRTMIN; i <= SIGRTMIN + 5; i++) {
            // Назначение обработчика
            if (sigaction(i, &act, NULL) == -1) {
                printf("Ошибка sigaction\n");
                exit(1);
            }
        }
        // Бесконечный цикл ожидания
        while (true);
    } else {

```

```

        // Время на установку обработчика
        sleep(1);
        // Цикл отправки сигнала в реальном времени дочернему
        процессу
        for (int i = SIGRTMIN; i <= SIGRTMIN + 5; i += 2) {
            printf("Отправка сигнала PB: %d, pid = %d, ppid = %d\n",
i - SIGRTMIN, getpid(), getppid());
            kill(child, i);
        }
        for (int i = SIGRTMIN + 5; i >= SIGRTMIN; i -= 2) {
            printf("Отправка сигнала PB: %d, pid = %d, ppid = %d\n",
i - SIGRTMIN, getpid(), getppid());
            kill(child, i);
        }
        printf("\n");
        sleep(1);
        // Убийство дочернего процесса
        kill(child, SIGKILL);
        wait(NULL);
    }

}

```

Вывод программы:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/6$ gcc 6.c -o 6
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/6$ ./6

```

```

Родительский процесс отправил сигнал PB: 0
Родительский процесс отправил сигнал PB: 2
Родительский процесс отправил сигнал PB: 4
Родительский процесс отправил сигнал PB: 5
Родительский процесс отправил сигнал PB: 3
Родительский процесс отправил сигнал PB: 1

```

```

Дочерний процесс поймал сигнал PB: 5
Дочерний процесс поймал сигнал PB: 4
Дочерний процесс поймал сигнал PB: 3
Дочерний процесс поймал сигнал PB: 2

```



```
Дочерний процесс поймал сигнал PB: 1
Дочерний процесс поймал сигнал PB: 0
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/6$
```

Из логов видно, что независимо от порядка отправки сигналов, сигналы с меньшими номерами доставляются раньше сигналов с большими номерами, т.е. сигнал с номером SIGRTMIN имеет «большой приоритет», чем сигнал с номером SIGRTMIN+1.

7. Задание: Используя функцию `sigaction()`, продемонстрируйте возможности управления линейкой сигналов, включая собственные обработчики и маскирование для разных сигналов, а также вариативность, предоставляемую структурами `sigaction (act / oldact)`.

Решение:

Функция `int sigaction( int sig, const struct sigaction * act, struct sigaction * oact)` позволяет вызывающему процессу получить информацию или установить (или и то и другое) действие, соответствующее какому-либо сигналу или группе сигналов.

`Sig` - тип сигнала, `act` – ненулевое значение отвечает за изменение действия при указанном сигнале, `oact` - ненулевое значение сохраняет предыдущее действие при указанном сигнале в структуре типа `sigaction`, на которую указывает указатель `oact`. Комбинация `act` и `oact` позволяет запрашивать или устанавливать новые действия при поступлении сигнала.

Sigaction() гарантирует надежную передачу, т.е. если при возникновении сигнала система занята обработкой другого сигнала (назовем его «текущим»), то возникший сигнал не будет потерян, а его обработка будет отложена до окончания текущего обработчика.

Была написана программа, передопределяющая обработчики сигналов SIGINT и SIGQUIT, маскирующая сигналы SIGQUIT и SIGUSR1, вызывающая эти три сигнала, выводящая информацию о них и статус процессов.

Исходный код: Файл 7.c

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Обработчик сигнала для SIGINT
void handlerSIGINT(int signum) {
    printf("Обработчик SIGINT\n");
    system("ps -s");
    exit(0);
}

// Обработчик сигнала для SIGQUIT
void handlerSIGQUIT(int signum) {
    printf("Обработчик SIGQUIT\n");
}

int main() {
    // Структура для старого обработчика сигнала
    struct sigaction old_act;
    // Структура для нового обработчика сигнала
    struct sigaction act;
```

```

// Сброс маски сигналов
sigemptyset(&act.sa_mask);
// Не используем специальных флагов при обработке сигнала
act.sa_flags = 0;

sigset_t mask1;
sigset_t mask2;
sigemptyset(&mask1);
sigemptyset(&mask2);

// Установка обработчика сигнала для SIGINT
// Задание пользовательской функции обработки сигнала
act.sa_handler = handlerSIGINT;
// Связываем обработчик сигнала SIGINT с обработчиком act и
old_act
// В случае ошибки
if (sigaction(SIGINT, &act, &old_act) == -1) {
    printf("Ошибка sigaction\n");
    exit(1);
}

// Установка обработчика сигнала для SIGQUIT
// Задание пользовательской функции обработки сигнала
act.sa_handler = handlerSIGQUIT;
// Связываем обработчик сигнала SIGQUIT с обработчиком act
// В случае ошибки
if (sigaction(SIGQUIT, &act, NULL) == -1) {
    printf("Ошибка sigaction\n");
    exit(1);
}
// Маскирование SIGQUIT (4)
// добавляем SIGQUIT в маску
sigaddset(&mask1, SIGQUIT);
// сигналы в маске будут заблокированы
if (sigprocmask(SIG_BLOCK, &mask1, NULL) == -1) {
    printf("Ошибка sigprocmask\n");
    exit(1);
}

```

```

// Маскирование SIGUSR1 (200)
// добавляем SIGUSR1 в маску
sigaddset(&mask2, SIGUSR1);
if (sigprocmask(SIG_BLOCK, &mask2, NULL) == -1) {
    printf("Ошибка sigprocmask\n");
    exit(1);
}

// Вывод информации об обработчиках сигналов
printf("SIGINT: обработчик %p; старый обработчик %p, маска
NULL\n", handlerSIGINT, old_act.sa_handler);
printf("SIGQUIT: обработчик %p; старый обработчик NULL;
маска ", handlerSIGQUIT);
for (int i = 1; i <= 32; i++) {
    if (sigismember(&mask1, i)) {
        printf("%d ", i);

    }
}
printf("\n");
printf("SIGUSR1: обработчик SIG_DFL; старый обработчик
NULL; маска ");
for (int i = 1; i <= 32; i++) {
    if (sigismember(&mask2, i)) {
        printf("%d ", i);

    }
}
printf("\n");

/*    // Вызываем сигналы с задержкой
printf("\nВызов сигнала SIGQUIT\n");
kill(getpid(), SIGQUIT);
system("ps -s");
sleep(1);*/

printf("\nВызов сигнала SIGUSR1\n");
kill(getpid(), SIGUSR1);
system("ps -s");

```

```

        sleep(1);

        printf("\nВызов сигнала SIGINT\n");
        kill(getpid(), SIGINT);
        sleep(1);

        return 0;
}

```

Вывод программы:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/1$ ./1
```

```
SIGINT: обработчик 0x55bbc33032e9; старый обработчик (nil), маска NULL
```

```
SIGQUIT: обработчик 0x55bbc3303320; старый обработчик NULL; маска 3
```

```
SIGUSR1: обработчик SIG_DFL; старый обработчик NULL; маска 10
```

Вызов сигнала SIGQUIT

```
UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND
```

```
1000 14203 0000000000000000 0000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
```

```
1000 14706 0000000000000000 0000000000010204 0000000000000006 0000000000000000 S+ pts/0 0:00 ./1
```

```
1000 14707 0000000000000000 0000000000000000 0000000180000000 0000000000010002 S+ pts/0 0:00 sh -c ps -s
```

```
1000 14708 0000000000000000 0000000000000000 0000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s
```

Вызов сигнала SIGUSR1

```
UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND
```

```
1000 14203 0000000000000000 0000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
```

```
1000 14706 00000000000000200 0000000000010204 0000000000000006 0000000000000000 S+ pts/0 0:00 ./1
```

```
1000 14709 0000000000000000 0000000000000000 0000000180000000 0000000000010002 S+ pts/0 0:00 sh -c ps -s
```

```
1000 14710 0000000000000000 0000000000000000 0000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s
```

Вызов сигнала SIGINT

```
Обработчик SIGINT
```

```

UID PID PENDING BLOCKED IGNORED CAUGHT STAT TTY TIME COMMAND
1000 14203 000000000000000000 00000000000010000 0000000000384004 000000004b813efb Ss pts/0 0:00 bash
1000 14706 00000000000000200 00000000000010206 00000000000000006 00000000000000000 S+ pts/0 0:00 ./1
1000 14711 000000000000000000 00000000000000000 00000000180000000 00000000000010002 S+ pts/0 0:00 sh -c ps -s
1000 14712 000000000000000000 00000000000000000 00000000180000000 0000000073d1fef9 R+ pts/0 0:00 ps -s

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/1$

```

Опытным путем было выяснено, что сигналу SIGUSR1 соответствует значение «200», а SIGQUIT - «4» в столбцах вывода статуса процессов. В столбце PENDING видны оба этих числа, что говорит о том, что с помощью маскирования SIGQUIT и SIGUSR1 были действительно заблокированы. Отсутствие вывода сообщения о запуске обработчика SIGQUIT также подтверждает этот факт. А сработавший пользовательский обработчик SIGINT вывел соответствующее сообщение и завершил работу программы.

Кроме того, в выводе указаны адреса пользовательских обработчиков SIGINT и SIGQUIT, адрес предыдущего обработчика SIGINT и маски SIGQUIT и SIGUSR1. Значения, помеченные NULL, говорят о том, что эти поля просто не рассматривались в программе, но были оставлены для красоты вывода.

Выводы.

Изучены межпроцессные взаимодействия в Unix-подобных ОС. Команды и утилиты для взаимодействия для межпроцессных взаимодействий Linux (в частности, sig, pipe, fifo, msg), изучены и успешно применены на практике.

Использованная литература:

Методическое пособие Душутиной Е.В. «Межпроцессное взаимодействие в операционных системах»